

予約リマインダー・フォローメッセージ 送信動作 監査レポート（Codex）

作成日：2026-09-11（JST）／監査者：Codex

対象：各店舗ページ「店舗情報編集」の予約リマインダー／フォローメッセージ、設定保存API、予約作成・変更・取消、Supabase Edge Functions、cron定義。

対象コミット：e36b2bdf047780a031760030933deec5e0a0d7b5。監査時のワークツリーを確認。既存のClaude Codeレポートは別資料として照合し、上書きしていない。

1. 結論

現状を「設定どおり確実に送られるので問題なし」とは判定できない。日付計算の基本部分は正しいが、送信漏れ・設定と実行のずれ・二重送信につながる実装上の問題を確認した。

とくに「何日前」「送信時刻」の相談に直接関係するのは次の4点である。

- リマインダーは指定した日の指定時間帯に実行されなければ、その予約を取り逃す。翌時間・翌日の自動回収はない。指定時刻の処理後に予約を登録・変更した場合も対象から漏れる。
- フォローの既存配信予定は、店舗の「何日後・基準日・時刻」を変更しても更新されない。保存された旧日時で送られる。文面と有効／無効は送信時の設定を読むため、設定項目によって反映のタイミングが異なる。
- フォローのcron定義は毎時05分。画面に「12:00に送信」と表示されても、通常の起動は12:05であり、全件が12:00に送られる仕組みではない。
- 本番のcron登録、認証、デプロイ内容、実際の店舗設定・送信履歴は未確認。コード中には仮の認証キーを含むcron定義があり、運用設定による全件不送信の可能性も残る。

さらに、リマインダーの重複実行防止がないこと、フォローの排他制御が不十分なこと、通信例外で後続処理が止まることを実行再現した。**本番運用の確認と、コードの修正の両方が必要**である。

2. 調査範囲と証拠の強さ

区分	実施内容・限界
静的監査	UI→店舗保存→予約登録／編集／LINE取消→配信予定→送信Function→cronの経路を追跡
既存テスト	関連4ファイル・41件すべて成功
追加再現	実際の送信Functionと予定管理コードを読み込み、時刻・DB・LINE通信を模擬した14シナリオすべて再現

区分	実施内容・限界
本番／staging	DB接続設定・専用接続ツールがこの作業環境にない。実データ、cron、認証成否、稼働Functionのバージョンは独自確認できていない
LINE実送信	実施していない。顧客へのメッセージは送っていない
変更範囲	レポートと再現用資料のみ。アプリ・DB・cron・既存レポートは変更していない

「再現済み」は条件を満たした場合のコード動作を確認したという意味であり、特定店舗で実際に発生した証拠ではない。追加再現のPASSは「問題を再現できた」を含み、安全性の合格判定ではない。モックは実DBの制約・RLS・取得上限・実ネットワークを再現しない。

3. 設定から送信までの実際の動作

3.1 予約リマインダー

画面の選択肢は1～7日前、07:00～21:00の毎正時。店舗オブジェクトをPUTし、DBの店舗行に保存する。

cronがFunctionを起動すると、JSTの現在時刻を「HH:00」に丸め、`reminder_enabled = true`かつ`reminder_time = HH:00`の店舗を検索する。店舗ごとに「今日＋何日前設定」の予約日を求め、その日付の未キャンセル予約へ送信する。複数店舗の対象日は、送信ループで店舗別に再照合する。

例：予約が9月20日、3日前・19:00なら、9月17日19時台に起動したFunctionが対象にする。「予約の72時間前」ではなく、**予約日の3暦日前の19:00 JST**である。正常起動時のこの日付計算に誤りは見つからなかった。

根拠：`src/app/tenant/[tenantSlug]/admin/[storeId]/page.tsx:1060,2606,2623`、`src/app/api/stores/[storeId]/route.ts:135`、`supabase/functions/send-reminders/index.ts:10,151,166,194,274`。

3.2 フォローメッセージ

予約作成時、店舗設定がONでLINE IDがあり、予定日時が未来であれば、`follow_messages`を作る。基準日は来店日、または予約受付日時をJSTに変換した日付。基準日＋何日後の指定時刻をUTCの`scheduled_at`として保存する。

cronは毎時05分に起動する定義。送信時には`status = scheduled`かつ`scheduled_at <=` 現在を古い順に最大200件取得する。店舗OFF、トークンなし、元予約取消、次回予約あり、リマインダー同日の順に見送り判定し、送信結果を記録する。

例：9月10日の来店＋7日・12:00なら保存値は9月17日03:00 UTC（12:00 JST）。定義どおりのcronによる最初の処理は9月17日12:05 JSTとなる。

根拠：`src/lib/follow-message-scheduler.ts:46,64`、`src/lib/follow-message-repository.ts:89,116`、`src/app/api/reservations/route.ts:600,932`、`supabase/functions/send-follow-messages/index.ts:187`。

4. 優先して対応すべき問題

重要度「高」は送信漏れ・重複・誤日程が発生しうるもの、「中」は表示との不一致、限定条件の問題、運用上の追跡不足とした。

A01【高・再現済み】リマインダーに取り逃し回収がない

症状：指定日に送られず、その後も届かない。

現在の時間帯と予約日が完全一致するものしか取得せず、未送信の配信予定を保持していない。たとえば9月12日予約・前日12:00の実行が止まり、9月11日13:00に復旧しても対象外。9月12日12:00は9月13日予約を検索するため、前日の漏れを回収できない（再現R3）。

同じ問題は、処理が終わった後の予約登録、予約日変更、OFFからONへの変更、過ぎた時刻への設定変更、3日前設定の店舗に2日前から入った予約でも生じる。時刻変更を契機に同日もう一度対象になれば、A02の重複にもつながる。

対策：予定日時と送信状態をDBに持ち、期限を過ぎた未送信分を回収する。予約時刻を過ぎてまでリマインドしないよう、回収期限も定義する。

根拠：send-reminders/index.ts:166,194,203,274。再現R3。

A02【高・再現済み】リマインダーは同じ予約へ複数回送信できる

送信済み記録、排他取得、LINEのリトライキーがない。同じ時間帯にFunctionを2回呼ぶと、同じ予約へ2回送信する（R2）。手動再実行、別名の重複cron、再送処理、送信後の時刻・日数変更が契機となる。通常cronが1本あることだけでは、重複防止を保証しない。

対策：予約＋通知種別＋対象日程を識別できる一意キー、配信状態、送信処理の排他取得を設ける。LINEへの初回送信から同じ配信のリトライキーを付ける。

根拠：send-reminders/index.ts:130,268,297,303。再現R2。

A03【高・再現済み】フォローの設定変更が既存予定に反映されない

店舗PUTは店舗行だけを更新し、既存follow_messagesを再計算しない。送信Functionも店舗のfollow_days_after / follow_time / follow_baseを取得しない。

例：9月1日＋7日・12:00の旧予定が残った状態で30日後・21:00へ変更しても、旧予定が期限到来なら送信される（F1）。文面は新設定を読むので「新しい文面が、旧日程で届く」こともある。

対策：設定変更を既存の未送信予定にも適用するか、新規予約だけに適用するかを明示する。現UIではこの差を説明していない。既存分にも適用するなら変更時の再計算と、すでに過ぎた予定の扱いを実装する。

根拠：src/app/api/stores/[storeId]/route.ts:185、src/lib/follow-message-repository.ts:99,276、send-follow-messages/index.ts:193,214。再現F1。

A04【高・再現済み】フォローの樂觀ロックでは送信中の再取得を防げない

attempt_countを条件付きで増やすだけで、状態はscheduledのまま。同じ古い値を読んだ2処理の競合は防げるが、後続処理が増加後の値を読めば再度取得できる。

再現F2：実行Aが0→1へ更新後、LINE応答待ちに入る。その間に実行Bを開始すると1を読み、1→2の更新に成功し、両方が送信する。DBの原子的な条件付き更新自体が正しくても起こる競合である。

通常の毎時cronで常に発生するという意味ではないが、手動実行・重複ジョブ・障害復旧時の重なりには耐えない。送信成功後のDB更新失敗でも、次回に同じメッセージを再送しうる。

対策：sending状態または所有者・有効期限を持つリースによる原子的な取得、送信結果更新時の所有者照合、LINEリトライキーを導入する。

根拠：send-follow-messages/index.ts:193,297,331。再現F2。

A05【高・再現済み】通信例外で後続配信が止まり、フォローの再試行上限も破れる

両FunctionはLINEのfetch()やres.text()を予約ごとのtry/catchで囲んでいない。ネットワーク例外が発生するとその実行全体が中断し、残りの予約を処理できない（R5）。リマインダーには翌回の回収もない。

フォローは送信前に試行回数を増やすが、failedへの変更はHTTPレスポンスを受け取った非成功時にしか行わない。取得条件に回数制限もないため、通信例外が続くと4回目以降もscheduledのまま再試行する（F3）。一方、HTTP 500が返る通常エラーは3回でfailedとなる（F5）。

対策：予約ごとの例外処理、通信タイムアウト、エラー種別と回数の永続化、取得時の試行上限、バックオフを実装する。1件の異常を後続全件へ波及させない。

根拠：send-reminders/index.ts:135,297、send-follow-messages/index.ts:159,303,328,339。再現R5／F3／F5。

A06【高・再現済み】再予約を取り消しても、元のフォローは復活しない

新しい予約を作ると、同じ店舗・LINE IDの旧予定はsupersededになる。新予約をキャンセルしても新予約の行をcancelledにするだけで、元の行を戻さない。

再現Q1：予約A→予約B→Bを取消、の結果はA=superseded、B=cancelled。有効なAが残っていても、どちらのフォローも送られない。

対策：取消・日程変更時に、その顧客の有効予約から配信対象を再選択する。復活させない仕様を採用する場合は店舗への説明が必要。

根拠：src/lib/follow-message-repository.ts:131,180,234。再現Q1。

A07【高・コード確認】キュー作成失敗や競合で配信予定が欠落・重複する

予約保存とフォロー予定作成は別処理。予定作成が失敗しても関数はnullを返し、予約作成APIは予約成功として返す。再作成ジョブは見つからない。

さらに「旧予定をsupersededにする」「既存行を読む」「新予定を作る」が別々のDB操作で、トランザクション化されていない。旧予定の取消後にINSERTが失敗すれば有効な予定がなくなる。異なる予約の同時作成では両方がscheduledになる可能性もある。予約単位の一意制約だけでは、同一顧客の複数行を防げない。

対策：予約保存と配信要求を一体で永続化する仕組み、顧客単位の原子的な差し替え、欠損検出と再作成を用意する。DB競合そのものは今回のモックで負荷再現しておらず、静的監査によるリスク判定。

根拠：src/app/api/reservations/route.ts:898,932、src/lib/follow-message-repository.ts:180,191,212,223、20260908000000_add_follow_message.sql:51。

A08【高・本番要確認】cron・認証・デプロイに運用上の未確認点

初期リマインダーcronは0 10 * * * (19:00 JST)。更新用は0 * * * *、フォローは5 * * * *だが、どちらも認証値がBearer <SERVICE_ROLE_KEY>というプレースホルダーで、呼出先URLも特定プロジェクト固定である。

毎時更新を実行していなければ古い毎日実行の可能性がある。一方、**ファイルをそのまま実行して成功すれば毎時cronは登録されるが、仮キーのままになる**。「手動登録していない＝必ず毎日実行のまま」とは言えない。pg_cron未導入なら登録はスキップされる。既存ジョブがない状態でのcron.unschedule失敗にも注意が必要。

stagingにも同じSQLを適用すると、固定された呼出先を意図せず共有するおそれがある。プロジェクト対応、認証、Functionの配備、必要な列のマイグレーション適用を確認する必要がある。本番でどの条件に該当するかは未確定であり、原因の確率順位まではつけられない。

根拠：20250102000000_prod_cron_job.sql:14、20260411100001_update_cron_hourly.sql:9、20260908000000_add_follow_message.sql:79。確認SQLは第8章。

5. その他の問題・仕様上の注意

B01【中・確定】フォローは設定時刻より通常5分以上遅れる

UIは正時送信と説明するが、cronは05分起動。さらに1回200件、LINEへの逐次送信なので、後続行は遅れる。201件の有効な期限到来行で、1件が次回に残ることをF4で再現した。毎時実行なら次回は約1時間後。大量滞留時には日付もずれうる。

対策：実行頻度、処理件数、処理能力を整え、画面には配信時間の幅を説明する。根拠：send-follow-messages/index.ts:25,198,274、cron SQL:84、店舗画面:2745。

B02【中・再現済み】予約内容編集で、送信待ちフォローが消える

予約編集APIは日時だけでなくメニュー・オプション・スタッフ等の編集でも再計算する。その際、計算後の予定が現在以前ならskipped / store_disabledにする。

例：12:00予定、12:05配信待ちの12:01に内容を編集すると、予定は「過去」になり配信されない（Q2）。店舗が有効でも理由は「店舗無効」であり、調査を誤らせる。障害で滞留中の予約編集も同様。

対策：日時に影響しない編集では配信予定を変えない。期限到来分を保持し、過去日時・LINE ID欠落・店舗OFFの理由を分ける。根拠：予約編集API:252,66、予定repository:99,288,303。

B03【中・仕様】ON前の予約にはフォローを後付けしない

ON前の予約、LINE IDなし、過去の予定には行を作らない。ONに切り替えてから予約内容を編集しても、既存行がなければ作成しない（Q3）。ただしキャンセル解除は新規予定作成経路を呼ぶので、条件を満たせば対象になる。既存レポートの「ON後に成立した予約だけ」にはこの例外がある。

また送信時にOFF／トークンなしだった予定はskippedという終了状態になり、後からONやトークン修正をしても自動復活しない。OFF期間中に送信時刻が来なければ、ONへ戻した後に残存予定が送られうる。根拠：repository:95,284、予約編集API:56、送信Function:280。

B04【中・再現済み／コード確認】未確定・来店前のメッセージ

リマインダーはpendingにも送る（R6）。フォローもcompletedを要求せず、取消以外を許可する。希望日時方式では、reservation_dateに保存された第一希望日を基準にするため、第二希望へ決まった場合は実際の予約日を更新する必要がある。ステータスだけの変更では日時は書き換わらない。

フォローを「予約受付日から」にすると、来店前にデフォルトの「ご来店ありがとうございました」を送ることもある。たとえば9月1日受付・9月20日来店・7日後設定は9月8日配信。基準日の選択自体は仕様だが、文面と対象条件を合わせる必要がある。

根拠：send-reminders/index.ts:210、send-follow-messages/index.ts:74,287、src/lib/static-generator-reservation.tsのreservation_date: selectedDate、src/lib/follow-message-scheduler.ts:51。

B05【中・再現済み】リマインダー設定のサーバー検証不足

店舗保存APIはフォロー設定を検証するが、同等のリマインダー検証がない。通常UIはHH:00だけを選べるものの、API・既存データ経由で12:30や19:00:00が保存されると毎時の照合に一致しない（R4）。不正な日数はFunction側で1日前に置き換えられるため、保存値と送信日数が食い違う。

対策：有効／無効、日数、時刻、テンプレート型を保存時に検証し、DB制約も合わせる。根拠：店舗保存API:13,145,185、送信Function:22,168。

B06【中・条件付きリスク】取得上限・処理時間・監視

リマインダーの店舗・予約取得とフォローの再予約照合にはページングがない。実環境のAPI取得上限を超えると予約が欠けたり、次回予約あり判定を取り逃したりする可能性がある。Supabaseは既定で最大1000行を返すが、本番の設定値は未確認。[Supabase select仕様](#)

cron SQLはnet.http_postのタイムアウト指定を省略している。実環境のデフォルト、Function処理時間、HTTP応答を照合すべきであり、「10件超なら必ず5秒超」「呼出側タイムアウト＝Function停止」とは判断できない。pg_netのレスポンスは標準設定では6時間保持のため、履歴が空でもcron停止とは限らない。[pg_net公式資料](#)

また個別LINE送信失敗があっても両Functionは通常HTTP 200を返す。リマインダーではerrors、フォローではfailedやDB状態も見る必要がある。cronのSQL実行成功はLINE送信成功を意味しない。

B07【中・外部仕様確認】LINE API成功と顧客への到達は別

ブロック等の条件ではLINE Push APIが200でも受信されない。現実装のsentはAPI受付成功を意味し、受信・既読の保証ではない。トークンなし、無効なトークン、LINE ID欠落、送信上限等も別途確認が必要。空トークンのリマインダーは黙って除外される。

LINEは重複リクエスト防止のX-Line-Retry-Keyを提供するが、両Functionとも付与していない。保持期間等の制約もあるためDB上の状態管理と併用する。[LINE Push API仕様](#)、[LINE再試行仕様](#)

B08【中・コード確認】見送り判定・キャンセルの境界

フォローの「リマインダー同日」は実際の送信成功を確認せず、日付条件だけで見送る。リマインダー側が故障している場合は両方届かない可能性がある。遅延したフォローは実行日の同日判定を使い、元の予定日の判定とは異なる。

両Functionとも取得後から送信までに予約取消・設定OFFが発生した場合、取得済み情報で送る可能性がある。フォローは取得時点での取消等を確認できるが、送信直前までの全競合を排除する仕組みではない。

なお次回予約判定は「同じ店舗・同じLINE ID・基準日より後」であり、送信日より先の予約も対象。これは既存設計に明記されている。同一日再来店は日付比較上対象外で、手動予約でLINE IDが異なる／ない場合は同一人物でも認識できない。

根拠：send-follow-messages/index.ts:50,61,244,280、src/lib/follow-message-scheduler.ts:87。

6. 正常と確認できた点

観点	確認結果
JSTと日付境界	リマインダーの月末・年末・うるう年・UTC/JST境界を実コードで確認。1/3/7/30日前の計算は期待どおり
フォロー日時	来店日／受付日を基準にしたUTC保存値、月・年または既存テストで確認
店舗別の照合	店舗ごとにリマインダー対象日を再照合。フォローの顧客照合も店舗ID+LINE ID
通常の取消	管理画面APIとLINE Webhookからフォロー取消関数を呼ぶ。送信側も取得時に元予約取消を検査
フォロー通常失敗	HTTP非成功が返る場合は3回でfailed。通信例外は別問題（A05）
文面	関連テンプレートテスト6件成功。ただしLINE本番での描画・文字数限界までの検証ではない
新規予定と日時編集	有効・LINE IDあり・未来予定なら予定を作成。未送信行の日付変更は再計算

正常経路の成功は、障害時・同時実行時を含む配信保証ではない。

7. Claude Codeレポートとの照合

既存レポートの記載	Codexの判定
日付の基本計算は正しい	同意。ただし設定変更や実行取り逃しを含めた「送信全体は問題なし」には同意できない
cronが古い／認証不一致が最有力	重要な確認候補。ただし実DB証拠がないため、最有力と順位付けする根拠は不足
db pushだけでは毎時にならない	訂正が必要。SQLが成功すれば毎時登録されるが、仮キーが残る。未適用と仮キー適用済みを分ける
フォローは問題なし・楽観ロックで二重送信防止	A03～A07、B01～B03の問題あり。送信中の後続起動で二重送信を再現
失敗は3回で終了	HTTP非成功では正しいが、通信例外では4回以上再試行する
HTTP 200なら正常	個別送信失敗を含む200や、LINE受付成功でも未到達のケースを区別する必要あり
pg_net応答がなければcronが動いていない	応答保持期間、別Functionの応答、呼出前の失敗もあるため断定できない
デプロイ版数・JWT検証設定を確認済み	Claude側の記載として扱う。Codexは独立確認していない

8. 本番で原因を特定するための読取SQL

以下は実行していない確認用SQL。認証キー、LINE ID、顧客名、メッセージ本文を出さず、状態と集計を確認する。本番／stagingそれぞれで実行する。cronの変更や再送は含めない。

8.1 cron登録・呼出先・プレースホルダー

```
select jobid, jobname, schedule, active,
       substring(command from 'https://[^/ ]+supabase[.]co') as target_origin,
       position('<SERVICE_ROLE_KEY>' in command) > 0 as placeholder_key,
       position('timeout_milliseconds' in command) > 0 as explicit_timeout
from cron.job
where command like '%send-reminders%'
      or command like '%send-follow-messages%'
order by jobid;
```

期待値：意図した各ジョブが1本ずつ、active=true。リマインダーは0 * * * *、フォローは現定義なら5 * * * *。同じURLを呼ぶ別名ジョブにも注意。仮キー判定falseだけで認証成功とは判断しない。

8.2 実行履歴とHTTP応答

```
select j.jobname, d.status,
       d.start_time at time zone 'Asia/Tokyo' as start_jst,
       d.end_time at time zone 'Asia/Tokyo' as end_jst
from cron.job_run_details d
join cron.job j on j.jobid = d.jobid
where j.command like '%send-reminders%'
      or j.command like '%send-follow-messages%'
order by d.start_time desc limit 100;

select id, status_code, timed_out,
       error_msg is not null as has_transport_error,
       created at time zone 'Asia/Tokyo' as created_jst
from net._http_response
order by created desc limit 100;

select extname, extversion from pg_extension
where extname in ('pg_cron', 'pg_net');
select name, setting from pg_settings where name like 'pg_net%';
select pg_get_function_arguments(p.oid) as http_post_arguments
from pg_proc p join pg_namespace n on n.oid = p.pronamespace
where n.nspname = 'net' and p.proname = 'http_post';
```

cronのsucceededは非同期HTTP要求の登録成功であり、Function／LINE成功ではない。HTTP応答テーブルは他の呼出も混ざるため、実行時刻・リクエストID・Functionログを対応付ける。401/403は認証経路、5xxはFunction等の異常を調査する。200でも応答本文の集計値とフォローDB状態を確認する。無関係な本文・秘密情報はレポートへ転記しない。

8.3 店舗設定と本日のリマインダー対象数

```
select id, name, reminder_enabled, reminder_days_before, reminder_time,
       coalesce(length(trim(line_channel_access_token)) > 0, false) as has_token,
       follow_enabled, follow_base, follow_days_after, follow_time
from stores order by name;

with settings as (
  select id, reminder_enabled, reminder_time,
         case when reminder_days_before between 1 and 30
              then reminder_days_before else 1 end as days_before
  from stores
)
select s.id as store_id, s.reminder_enabled, s.reminder_time,
       (now() at time zone 'Asia/Tokyo')::date + s.days_before as target_date,
       count(r.id) as eligible_reservations
from settings s
left join reservations r on r.store_id = s.id
  and r.reservation_date = (now() at time zone 'Asia/Tokyo')::date + s.days_before
  and r.status <> 'cancelled'
  and nullif(trim(r.line_user_id), '') is not null
group by s.id, s.reminder_enabled, s.reminder_time, s.days_before
order by s.id;
```

対象数は送信済み数ではない。必要列が存在しない場合はマイグレーションの適用状況を確認する。未到達の個別予約について、保存日付・作成時刻・設定変更の有無・期待送信日をJSTで照合する。過去の店舗設定履歴がなければ、現在値だけで過去の日程は断定できない。

8.4 フォロー状態・滞留・現在設定との不一致

```
select store_id, status, skip_reason, count(*)
from follow_messages
group by store_id, status, skip_reason
order by store_id, status;

select store_id, count(*) as overdue_count,
       min(scheduled_at) at time zone 'Asia/Tokyo' as oldest_due_jst,
       max(attempt_count) as max_attempt_count,
       count(*) filter (where attempt_count >= 3) as at_least_three_attempts
from follow_messages
where status = 'scheduled' and scheduled_at <= now()
group by store_id;

with expected as (
  select f.id, f.store_id, f.scheduled_at,
         ((case when s.follow_base = 'created_at'
              then (r.created_at at time zone 'Asia/Tokyo')::date
              else r.reservation_date end
          + case when s.follow_days_after between 1 and 60
              then s.follow_days_after else 7 end)
          + (case when s.follow_time ~ '^([01][0-9]|2[0-3]):00$'
              then s.follow_time else '12:00' end)::time)
         at time zone 'Asia/Tokyo' as current_setting_at
  from follow_messages f
  join stores s on s.id = f.store_id
  join reservations r on r.id = f.reservation_id
```

```
where f.status = 'scheduled'
)
select store_id, count(*) as different_from_current_setting
from expected where scheduled_at is distinct from current_setting_at
group by store_id;
```

不一致は旧設定固定の可能性を示す。意図した設定履歴なのか、運用で反映が必要なものを区別する。これらの集計だけでキュー欠損は完全検出できないため、具体的な未配信予約IDに対応する行の有無も確認する。

9. 推奨する修正順序と受入条件

- 1. 本番の事実確認**：第8章の読取結果とFunctionログで、ジョブ本数・時刻・接続先・認証・個別失敗を特定する。原因不明のままFunctionを手動連打するとA02/A04により重複送信のおそれがある。
- 2. 取り逃し・重複・通信障害への対処**：リマインダーの永続キュー、両Functionの排他取得と例外処理、LINEリトライキー、結果記録を実装する。
- 3. 設定変更と予約変更の整合性**：既存予定への反映方針、再予約取消後の復元、ON時の既存予約の扱い、キュー作成失敗の回復を実装・明示する。
- 4. 時間精度と可観測性**：フォローの05分ずれ、200件制限、取得上限、滞留通知、店舗別の送信結果表示を改善する。
- 5. 文面・対象条件**：未確定予約、希望日時、受付日起算の来店前お礼について運用とUI説明をそろえる。

修正後は、同じ時間帯の再実行で同一配信が増えない、停止後に期限内の未送信を回収できる、1件の通信例外が後続を止めない、ネットワーク例外でも試行上限が効く、設定変更後の既存予定が定義どおりになる、再予約取消後に適切な予定が残ることを確認する。最後にテスト専用店舗・受信アカウントで実送信し、JSTの予定／処理／API受付／受信を照合する。

10. 再現資料と実行記録

追加再現スクリプト：docs/codex-message-audit/reproduce.cjs。結果：docs/codex-message-audit/results.json。

```
node docs/codex-message-audit/reproduce.cjs
→ 14シナリオ成功（現状の動作・問題を再現）

node node_modules/vitest/vitest.mjs run
  src/lib/__tests__/follow-message-scheduler.test.ts
  src/lib/__tests__/follow-message-repository.local.test.ts
  src/lib/__tests__/line-message-template.test.ts
  src/lib/__tests__/reservation-custom-field-followup.test.ts
→ 4ファイル、41テスト成功（上記は実際には1行で実行）
```

再現ID	確認した内容	結果
R1	JST境界・月末・年末・うるう年、 1/3/7/30日前	正常計算
R2	同じ時間帯で2回実行	2回送信

再現ID	確認した内容	結果
R3	指定時間を逃し次時間／翌日実行	回収されない
R4	reminder_time=12:30	対象にならない
R5	最初のLINE通信で例外	残りを送らず中断
R6	pending予約	送信対象になる
F1	店舗設定を変えて旧予定を処理	旧予定で送信
F2	claim後・応答前に別実行	同じ予定を二重送信
F3	4回連続通信例外	試行4・scheduledのまま
F4	201件が期限到来	200件処理、1件残存
F5	HTTP 500を繰り返す	3回でfailed
Q1	予約A→B→B取消	AもBも配信予定なし
Q2	予定時刻後に予約内容再計算	skipped／store_disabled
Q3	OFF時予約→ON→予約編集	予定行を後付けしない

監査結果として、正常経路の計算は確認できたが、実装上の配信保証には複数の欠落がある。本番での個別未配信原因は未確定のため、上記の修正候補と運用確認を切り分けて進める必要がある。